201CP: OBJECT ORIENTED PROGRAMMING WITH C++

Prof. Tirth Patel

Computer Engineering Dept.

BVM Engineering College

Classes and Objects

Analogy – "Class as a Blueprint, Object as a House"

- Imagine you're an architect. You draw a blueprint of a house. That blueprint doesn't occupy land, doesn't have furniture — it's just a design.
- Now, if you build a house using that blueprint, that actual house is the *object*.
  - The **blueprint** = *Class*
  - The **actual house** = *Object*
- Similarly in programming:
  - A **class** defines how an object should look and behave.
  - An **object** is the real entity based on the class.

Contd.

Classes

- User-defined data types (a.k.a. Abstract Data Types)
- It is a template that represents a group of objects that share common properties and relationships.
- Once a class is defined, multiple objects of that class can be created
- Can be understood as a collection of objects as well
- Everything in C++ is associated with classes and objects, along with its attributes and methods.
- Attributes and methods are variables and functions (define the behavior/ action taken by an object) that belong to the class. These are often referred to as "class members".
- A class is a user-defined data type that we can use in our program, and it works as an object constructor or a "blueprint" for creating objects

Contd.

Example:

Class name

Attributes / data members

Methods / member functions

```
class Person {
public:
    string name;
    int age;

    void introduce() {
        cout << "Hi, I'm " << name << endl;
    }
};
```

Contd.

Objects

- Runtime entities
- Instances (variables) of class
- Are created from a class.
- May represent a real-world object (e.g. a person) or concept (e.g. an account)
- Contain data (attributes) and methods (member functions) to operate on data
- Interact without knowing details of each other. Only the interface is known

Principles of OOP

Encapsulation, Abstraction, Inheritance, Polymorphism

Contd.

```
Person p;                // object p of type Person (stack allocation)
p.name = "John";        // accessing data member
p.age = 25;
p.introduce();           // calls the method on this object
```

We use the dot operator (**.**) when working with an object, or the arrow operator (**->**) when using a pointer to object.

Contd.

Example:-

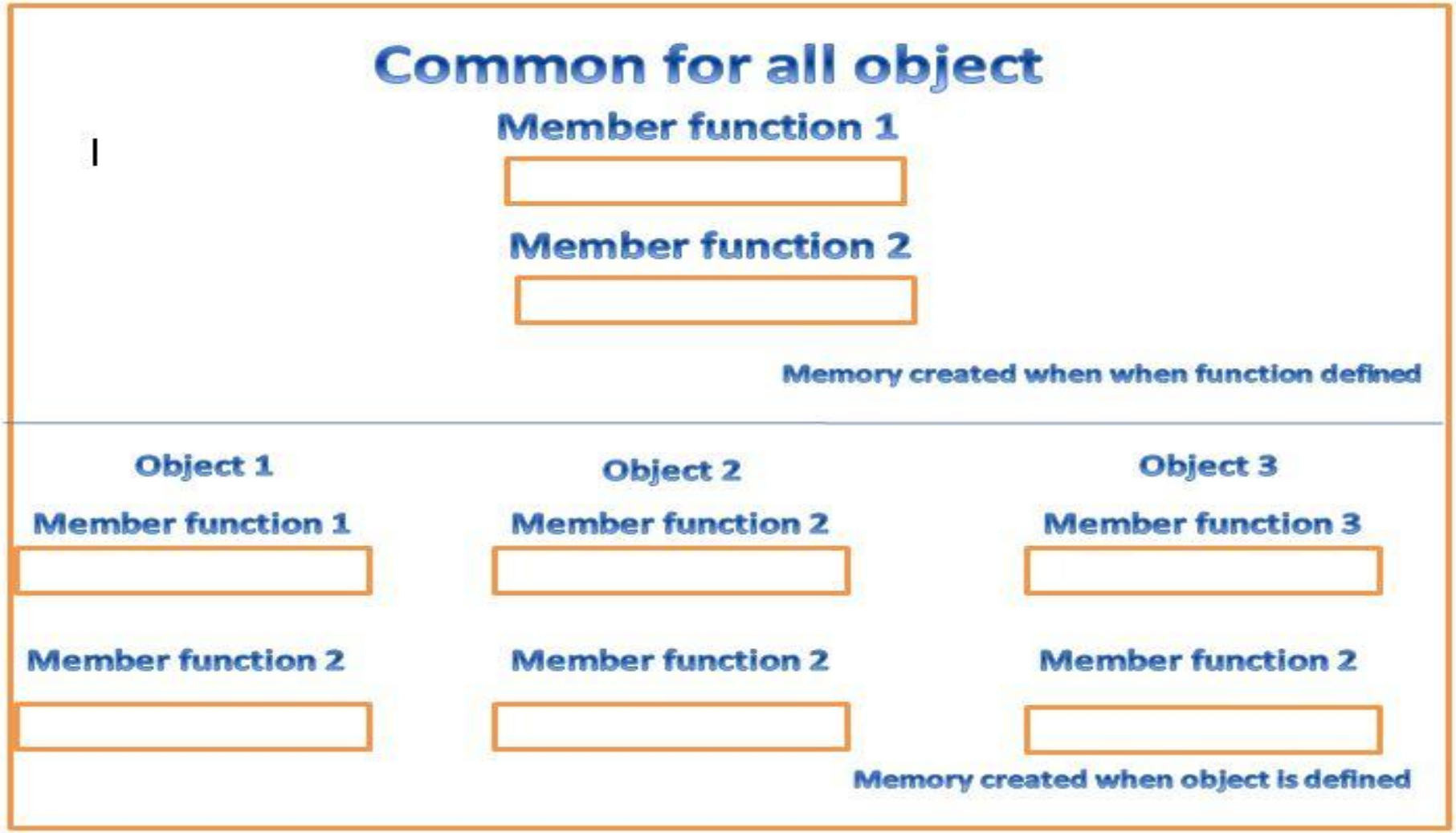
```
#include<iostream>
using namespace std;

// Class: Blueprint of a Student
class Student {
public:
    string name;
};
```

```
int main() {
    Student s1;  // Object created
    s1.name = "Neel";

    cout << "Student Name: " <<
s1.name << endl;

    return 0;
}
```

This is just a blueprint. No student exists yet.
We are only saying that every student will have a name.

Student s1; → creates an object of Student.
s1.name = "Neel"; → assigns value to that object.
Now, **s1** is a **real student** with a name.

MEMORY ALLOCATION FOR OBJECTS



Access specifiers — control who can use members

The three main specifiers:

- Public
  Accessible from anywhere (outside the class, in other functions, etc).
- Protected
  Accessible inside the class and in derived (sub)classes, but not from outside code.
- Private
  Accessible only inside the class itself (and friend functions/classes). Not accessible from outside or in derived classes.

Default visibility:
For **class**, default is **private**.
For **struct**, default is **public**.

Difference Between Class and Structure in C++

Aspect	Class	Structure
Default Access Specifier	private — members are private by default	public — members are public by default
Purpose	Generally used to model objects in OOP with encapsulation and abstraction	Traditionally used for grouping data together (C-style structs), but in C++ can have functions as well
Inheritance	Default inheritance is private	Default inheritance is public
Encapsulation	Strong encapsulation (members hidden by default)	Weak encapsulation (members visible by default)
OOP Features	Fully supports all OOP features — inheritance, polymorphism, access specifiers, constructors, destructors, etc.	Also supports OOP features in C++, but traditionally used for plain data storage
Keyword	class	struct
Usage Style	Used when data hiding and member functions are important	Used when a simple data container is needed without much restriction

What does private mean?

- Members (variables or functions) marked as private cannot be accessed directly from outside the class.
- They can only be accessed by:
  - Member functions of the same class
  - Friends (friend functions or friend classes)

Contd.

Without private:

```
class Account {
public:
    int balance;
};

int main() {
    Account a;
    a.balance = -1000; // Invalid
value, no protection!
}
```

With private:

```
class Account {
private:
    int balance;

public:
    void setBalance(int b) {
        if(b >= 0) balance = b;
    }

    int getBalance() {
        return balance;
    }
};
```

Now, balance cannot be set to a negative value from outside. You have **full control** over how data is modified.

Defining Member Functions

- A member function is a function that is declared inside a class and operates on the objects of that class. It has access to all members (data and functions) of the class, including **private** and **protected** members.
- Member functions help **encapsulate behavior** along with the data it acts on.

- A member function can be defined in **two ways**:

1. **Inside the Class Definition**
2. **Outside the Class Definition**

It is obvious that, irrespective of the place of definition, the function should perform the same task. Therefore, the code for the function body would be identical in both the cases. However, there is a subtle difference in the way the function header is defined. Both these approaches are discussed in detail in this section.

Contd.

1. Inside the Class Definition

Syntax:

```
class ClassName {
public:
    void functionName() {
        // function body
    }
};
```

Example:

```
class Person {
public:
    void introduce()
{
        cout << "Hello!" << endl;
    }
};
```

- The function body is written directly within the class.
- Such functions are implicitly inline.
- Useful for small, simple functions.

Contd.

2. Outside the Class Definition

Syntax:

```
class ClassName {
public:
    void functionName(); // only declaration
};

void ClassName::functionName() {
    // function body
}
```

definition outside

Example:

```
class Person {
public:
    void introduce();
};

void Person::introduce() {
    cout << "Hello!" << endl;
}
```

- The function is declared inside the class, but defined outside using the **scope resolution operator ::**
- Useful for larger functions to keep the class definition clean.

Example

```
class Rectangle
{
    private:
      int width, length;
    public:
      void set (int w, int l);
      int area() {return width*length; }
}
```

Inline

```
void Rectangle :: set (int w, int l)
{
    width = w;
    length = l;
}
```

class name

scope operator

member function name

```
Int main()
{
  Rectangle r1;
  r1.set(5,8);
  cout<<"area="<<r1.area();
  return 0;
}
```

Another Example

```
#include <iostream.h>

class circle
{
    private:
        double radius;

    public:
        void store(double);
        double area(void);
        void display(void);

};
```

```
// member function definitions

void circle::store(double r)
{
    radius = r;
}
double circle::area(void)
{
    return 3.14*radius*radius;
}
void circle::display(void)
{
    cout << “r = “ << radius << endl;
}
```

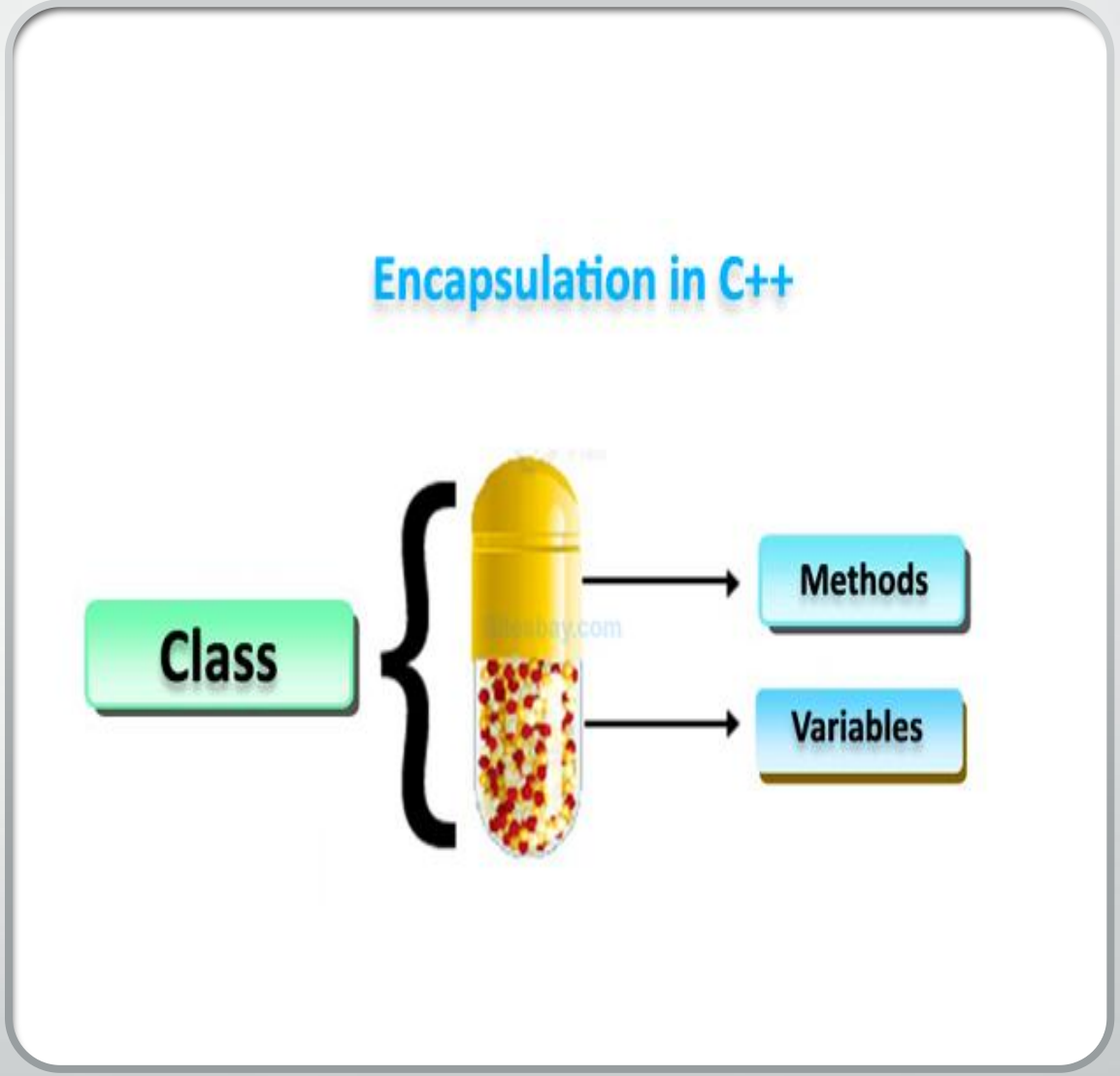
```
int main(void) {
    circle c;   // an object of circle class
    c.store(5.0);
    cout << "The area of circle c is " << c.area() << endl;
    c.display();
}
```

Differences between c and c++

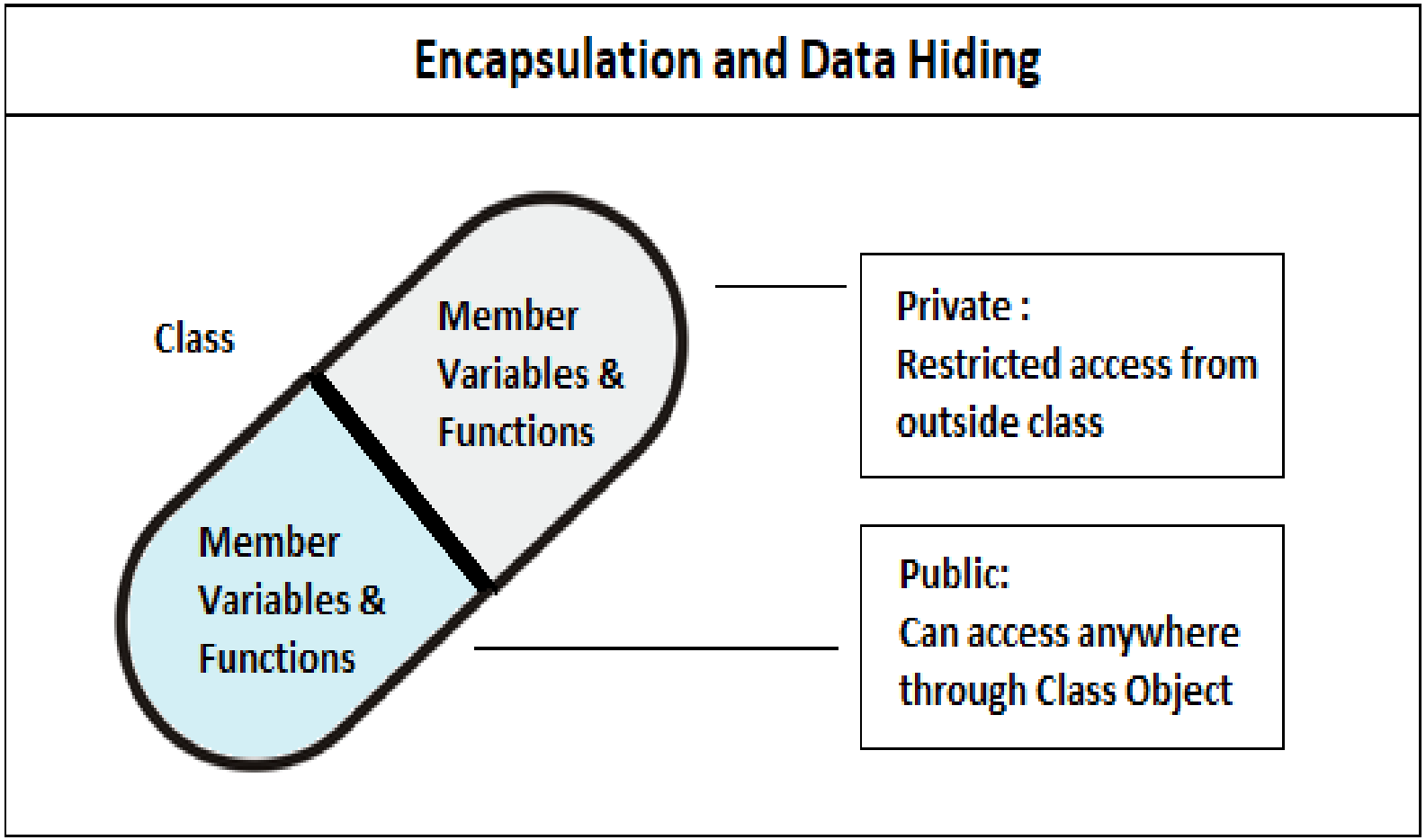
1. In c if function has no parameters then to write void
 in c++ it is optional in c++ following two declarations are equivalent
 char f1();
 char f1(void);
2. all the functions in c++ must be prototyped
3. in c++ if function is returning a value then must return. i.e. any return within the function must have a value. In c if does not then garbage value is returned
4. In c if you do not specify return type of function, an integer return type is assumed.in c++ you have to explicitly specify return type. in c++ you write int main()
5. In c All the local variables are declared in beginning. In c++ local variables can be declared anywhere.
6. c++ defines the bool data type, which is used to store value true/false

Encapsulation

- **Encapsulation** is the process of **binding data (variables)** and the **methods (functions)** that operate on that data into a **single unit (class),** and restricting direct access to some of the object's components.
- It helps in protecting the internal state of an object from unwanted external modification.
- **Data Hiding –** Restrict direct access to internal variables.
- **Control Access –** Allow access through public methods (getters/setters).
- **Maintainability –** Easy to change internal implementation without affecting outside code.
- **Security –** Prevent unintended data corruption or misuse.



Contd.


Encapsulation and Data Hiding
Class
Member Variables & Functions
Member Variables & Functions
Private :
Restricted access from outside class
Public:
Can access anywhere through Class Object

Encapsulation, Data Hiding, Abstraction, and Objects

- **Encapsulation** means putting data and the functions that work on that data together in one unit, usually a class. This helps keep things organized and safe.
- **Data Hiding** means that the data inside a class can't be directly changed or seen by code outside the class. Only the class's own functions can access or change the data. This protects the data from being misused.
- When data and functions are packed together like this, they form an **object**. You can think of an object like a **black box**—you can use it, but you don't need to know how it works inside.
- **Abstraction** refers to the act of representing essential features without including the back-ground details or explanation , in other words showing only the important details and hiding the unnecessary parts. It helps make complex things easier to understand.
- **Classes** use abstraction by showing only what's needed to use an object, while hiding the complex code behind it. That's why classes are called **Abstract Data Types (ADTs)**.

Contd.

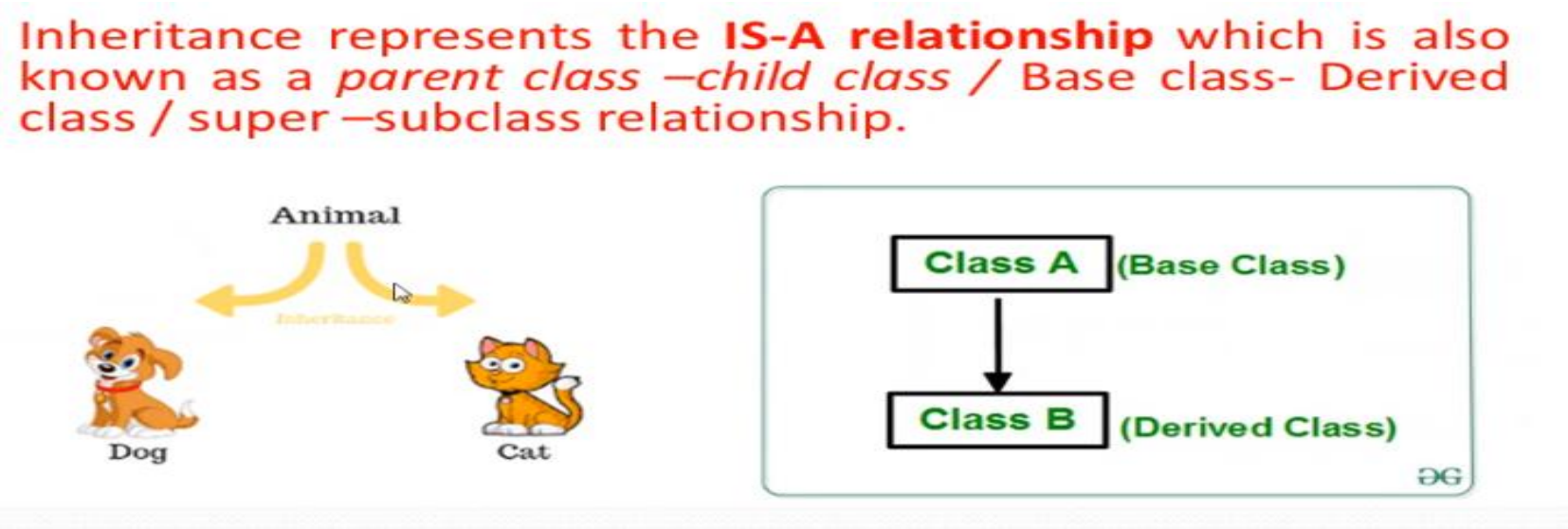
- Consider a real life example of a man driving a car,

The man only knows that pressing the accelerators will increase the speed of car or applying brakes will stop the car but he does not know about how on pressing accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of accelerator, brakes etc in the car. This is what abstraction is.

- **Abstraction using Classes:** We can implement Abstraction in C++ using classes. Class helps us to group data members and member functions using available access specifiers. A Class can decide which data member will be visible to outside world and which is not.

Inheritance

- **Inheritance** is a fundamental concept in object-oriented programming (OOP) that allows a class (called the **derived class** or **child class**) to acquire the properties and behaviors (data members and member functions) of another class (called the **base class** or **parent class**).



Contd.

This promotes **code reusability**, **extensibility**, and **modularity**.

- Code Reusability – Common code in base class is reused by derived class.
- Extensibility – You can extend the behavior of base class without modifying it.
- Maintainability – Easier to maintain and update the codebase.
- Data Hiding – Access specifiers support encapsulation of inherited data.

Contd.

- **Example:-**

```
// Base class (Parent)
class Animal {
public:
    void eat() {
        cout << "This animal eats food." << endl;
    }
};

// Derived class (Child)
class Dog : public Animal {
public:
    void bark() {
        cout << "The dog barks." << endl;
    }
};

int main() {
    Dog myDog;

    myDog.eat();   // Inherited from Animal
    myDog.bark();  // Own method

    return 0;
}
```

Polymorphism

- It means the ability to take more than one form.
- For example, a + is used to add two numbers, but it can also be used to concatenate two strings.
- This is known as operator overloading because the same operator may behave differently in different instances.
- Same way functions can be overloaded. E.g., sum() can be used to add two integers as well as two floating point numbers.
- Also known as early binding

Benefits of OOP

- Reusability of code
- Software complexity can be easily managed
- Division of work can be easily done based on classes
- Easy maintainability
- More secured programs
- Programming feels closer to real world

- There are many languages which support OOP concepts

- Today, OOP concepts are used to design most of the applications

NESTING OF MEMBER FUNCTIONS

- **Nesting of member functions** refers to the concept where **one member function of a class calls another member function of the same class**. This is a very common practice in object-oriented programming.

```
#include<iostream>
#include<string>


using namespace std;

class binary{

    string s;

public:

    void read(){

        cout<<"Enter a binary number:";
        cin>>s;
    }



    void chk_bin(){

        for(int i=0;i<s.length();i++){

            if(s.at(i)!= '0' && s.at(i)!='1'){

                cout<<"Incorrect binary number:";
                exit(0);
            }
        }
    }
```

```
void ones(){
        chk_bin();

        for(int i=0;i<s.length();i++){

            if(s.at(i)=='0'){

                s.at(i)='1';
            }
            else{

                s.at(i)='0';
            }
        }
    }


    void Displayones(){

        ones();
        cout<<"The 1's of the given
number is "<<s<<endl;
    }


};
```

```
int main(){
    binary b;
    b.read();
    b.Displayones();
    return 0;
}
```

Static Data Member

- A **static data member** is a class-level variable that is **shared among all objects** of that class. It is **not associated with any specific object**, and only **one copy** of it exists regardless of the number of objects created.

- **Key Characteristics:**
- Declared using the static keyword inside the class.
- Defined and initialized **outside the class** (usually in a .cpp file).
- Shared by all objects of the class.
- Memory is allocated **only once**, when the class is first loaded into memory.
- By default it is initialize to zero but can be given value at the time of definition
- Can be accessed using the **class name** as well as **through objects**.
- Its visible only within the class, but its **lifetime is the entire program**.

Syntax:

- Declaration (inside class) & Definition (outside class):

```
class ClassName {
public:
    static int staticVar;  // Declaration
};

// Static data member must be defined outside the class

int ClassName::staticVar = initial_value;  // Definition & Initialization
```

Why Do We Need Static Members?

1. To Share Common Data Across All Objects

Imagine a class Student. You want to count how many students (objects) have been created.

```
class Student {
public:
    static int count; // shared by all objects

    Student() {
        count++; // increment whenever an object is created
    }
};

int Student::count = 0;

int main() {
    Student s1, s2, s3;
    cout << "Total students: " << Student::count << endl; // Output: 3
    return 0;
}
```

Without static, each object would have its own count, and the total wouldn't be possible.

Contd.

2. **To Access Members Without Creating an Object**

```
class Utility {
public:
    static void sayHello() {
        cout << "Hello from Utility class!" << endl;
    }
};

int main() {
    Utility::sayHello(); // No object needed
}
```

3. To Save Memory

- Normal member variables: Each object gets a copy ⇒ more memory used.
- Static members: Only **one copy for the whole class** ⇒ memory efficient.

Contd.

4. **For Configurations or Constants Shared Across Objects**

You can define shared constants like interest rate, tax rate, etc., once and use across all objects.

```
class Bank {
public:
    static float interestRate;
};

float Bank::interestRate = 7.5;
```

Static Member Functions

- A **static member function** is a function that belongs to the **class** rather than any particular object of the class.
- It **does not operate on an instance** (object) of the class, and thus **cannot access non-static data members or non-static member functions** directly.
- Can be **called without objects .**
- No **this pointer -** Because it doesn't belong to an object.

Syntax:-

```
class ClassName {
public:
    static void staticFunction();  // Declaration
};
void ClassName::staticFunction() {
    // Code here
}
```

Example:

```
class Counter {
private:
    static int count;

public:
    static void increment() {
        count++;
    }

    static void display() {
        cout << "Count: " << count << endl;
    }
};

// Static data member must be defined outside the class
int Counter::count = 0;

int main() {
    Counter::increment();  // No object needed
    Counter::display();

    Counter c1, c2;
    c1.increment();
    c2.increment();

    Counter::display();  // Still accessible through class name
    return 0;}
```

Key Uses of Static Member Functions:

- **1. Access Static Data Members**

Static functions are primarily used to **access or manipulate static data members** of a class (which are shared across all instances of the class).

```
class MyClass {
    static int count;  // shared across all instances

public:
    static void incrementCount() {
        count++;
    }

    static int getCount() {
        return count;
    }
};
```

- Q. Can a static member function access non-static (normal) member variables?

- Q. Can a non-static (normal) member function access static member variables?

Answers

Q. Can a static member function access non-static (normal) member variables?

❌ No, it cannot.

Because static functions belong to the class, not to any specific object.They don't have access to the this pointer (which points to a specific object).Non-static (normal) member variables are tied to objects — and static functions have no object context.

Q. Can a non-static (normal) member function access static member variables?

✅ Yes, it can.

Because static variables exist at the class level, they are accessible from anywhere inside the class—including from object-level (non-static) functions.

Question 1: Predict Output or Fix Code

```
#include <iostream>
using namespace std;

class BankAccount {
    string accountHolder;
    static int totalAccounts;

public:
    static void openAccount(string name) {
        accountHolder = name;          // Line A
        totalAccounts++;               // Line B
        cout << "Account opened for " << accountHolder
<< endl;
    }
};

int BankAccount::totalAccounts = 0;
```

Answer:

❌ Error is in Line A:

accountHolder = name;

Reason: accountHolder is a non-static member, and openAccount() is a static function. Static functions do not have access to instance variables.

Question 2: Predict Output or Fix Code

```
#include<iostream>
using namespace std;

class Counter {
public:
    static int count;
    void increment() {
        count++;
    }
    static void display() {
        cout << "Current count: " << count << endl;
    }
};

int Counter::count = 0;

int main() {
    Counter c1, c2;
    c1.increment();
    c2.increment();
    Counter::display();
    return 0;}
```

Answer:

✅ Yes, the code will compile and run.

count is static, shared across all objects.

Each increment() call increases count.

Output:

```
Current count: 2
```

Array as data member

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
};

void change_name(char name[])
{
    name[0] = 'R';
}

void change_age(int age)
{
    age = 31;
}
```

```
int main()
{
    Person person1;
    strcpy(person1.name, "Sita");
    person1.age = 30;

    change_name(person1.name);
    change_age(person1.age);

    cout << person1.name << " ";
    cout << person1.age << endl;

    return 0;
}
```

Array as data member

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
};

void change_name(char name[])
{
    name[0] = 'R';
}

void change_age(int age)
{
    age = 31;
}
```

```
int main()
{
    Person person1;
    strcpy(person1.name, "Sita");
    person1.age = 30;

    change_name(person1.name);      // pass by pointer
    change_age(person1.age);        // pass by value

    cout << person1.name << " ";
    cout << person1.age << endl;

    return 0;
}
```

Rita 30

Passing Object by Value to Independent User-Defined Function

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
};

void change_name(Person person)
{
    person.name[0] = 'R';
}

void change_age(Person person)
{
    person.age = 31;
}
```

```
int main()
{
    Person person1;
    strcpy(person1.name, "Sita");
    person1.age = 30;

    change_name(person1);
    change_age(person1);

    cout << person1.name << " ";
    cout << person1.age << endl;

    return 0;
}
```

Passing Object by Value to Independent User-Defined Function

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
};

void change_name(Person person)
{
    person.name[0] = 'R';
}

void change_age(Person person)
{
    person.age = 31;
}
```

```
int main()
{
    Person person1;
    strcpy(person1.name, "Sita");
    person1.age = 30;

    change_name(person1);  // pass object by value
    change_age(person1);      // pass object by value

    cout << person1.name << " ";
    cout << person1.age << endl;

    return 0;
}
```

Sita 30

Passing Object by Reference to Independent User-Defined Function

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
};

void change_name(Person &person)
{
    person.name[0] = 'R';
}

void change_age(Person &person)
{
    person.age = 31;
}
```

```
int main()
{
    Person person1;
    strcpy(person1.name, "Sita");
    person1.age = 30;

    change_name(person1);
    change_age(person1);

    cout << person1.name << " ";
    cout << person1.age << endl;

    return 0;
}
```

Passing Object by Reference to Independent User-Defined Function

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
};

void change_name(Person &person)
{
    person.name[0] = 'R';
}

void change_age(Person &person)
{
    person.age = 31;
}
```

```
int main()
{
    Person person1;
    strcpy(person1.name, "Sita");
    person1.age = 30;

    change_name(person1);      // pass object by reference
    change_age(person1);       // pass object by reference

    cout << person1.name << " ";
    cout << person1.age << endl;

    return 0;
}
```

Rita 31

Array of Objects

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
};

void change_name(Person person[])
{
    person[0].name[0] = 'R';
}

void change_age(Person person[])
{
    person[0].age = 31;
}
```

```
int main()
{
    Person person[5];
    strcpy(person[0].name, "Sita");
    person[0].age = 30;

    change_name(person);
    change_age(person);

    cout << person[0].name << " ";
    cout << person[0].age << endl;

    return 0;
}
```

Array of Objects

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
};

void change_name(Person person[])
{
    person[0].name[0] = 'R';
}

void change_age(Person person[])
{
    person[0].age = 31;
}
```

```
int main()
{
    Person person[5];
    strcpy(person[0].name, "Sita");
    person[0].age = 30;

    change_name(person);        // pass object array by pointer
    change_age(person);         // pass object array by pointer

    cout << person[0].name << " ";
    cout << person[0].age << endl;

    return 0;
}
```

Rita 31

Array of Objects

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
    void print()
    {
        cout << name << " ";
        cout << age << endl;
    }
};

void change_name(Person person[])
{
    person[0].name[0] = 'R';
}

void change_age(Person person[])
{
    person[0].age = 31;
}
```

```
int main()
{
    Person person[5];
    strcpy(person[0].name, "Sita");
    person[0].age = 30;

    change_name(person);
    change_age(person);

    person[0].print();

    return 0;
}
```

Array of Objects

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
public:
    char name[50];
    int age;
    void print()
    {
        cout << name << " ";
        cout << age << endl;
    }
};

void change_name(Person person[])
{
    person[0].name[0] = 'R';
}

void change_age(Person person[])
{
    person[0].age = 31;
}
```

```
int main()
{
    Person person[5];
    strcpy(person[0].name, "Sita");
    person[0].age = 30;

    change_name(person);
    change_age(person);

    // We can access members of an element of object array

    person[0].print();

    return 0;
}
```

Rita 31

Independent functions cannot access private members

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
    int age;
public:
    char name[50];
    void print()
    {
        cout << name << " ";
        cout << age << endl;
    }

    void set_age(int age)
    {
        this->age = age;
    }

    int get_age()
    {
        return age;
    }
};
```

```
void change_name(Person person)
{
    person.name[0] = 'R';
}

void change_age(Person person)
{
    person.age = 31;
}

int main()
{
    Person person;
    strcpy(person.name, "Sita");
    person.set_age(30);

    change_name(person);
    change_age(person);

    person.print();
    return 0;
}
```

Independent functions cannot access private members

```
#include<iostream>
#include<cstring>

using namespace std;

class Person
{
    int age;
public:
    char name[50];
    void print()
    {
        cout << name << " ";
        cout << age << endl;
    }

    void set_age(int age)
    {
        this->age = age;
    }

    int get_age()
    {
        return age;
    }
};
```

```
void change_name(Person person)
{
    person.name[0] = 'R';
}
void change_age(Person person)
{
    person.age = 31;
}

int main(){
    Person person;
    strcpy(person.name, "Sita");
    person.set_age(30);
    change_name(person);

// error: ‘int Person::age’ is private within this context
    // change_age needs to be method or friend function
    // inorder to access its private members
    //change_age(person);

    person.print();
    return 0;
}
```

Friend Function

We can **allow outside functions** to **access private members** of a class

- To **make outside function friendly**, we need to **declare it as a friend within class definition**
  - With use of **friend** keyword
- Function **is defined outside the class definition**
  - Function **definition does not use keyword friend**
  - **Can't use class name and scope resolution operator** with definition
  - In **other words, function definition does not require any change**
    - Unlike methods of that class, **friend function can not even access private members without object name and dot operator**
- A function can be declared **as friend function in more than one class**
- It **can not be called using object of the class** in which it has be declared friendly. **Like definition, function call requires no change**
- It can be **declared in public or private area** without affecting its meaning

Example

```
#include <iostream>
using namespace std;

class sample {
    int a;
    int b;

public:
    void setvalue() {
        a = 25;
        b = 40;
    }
    friend float mean(sample s);
};

float mean(sample s) {
    return float(s.a + s.b) / 2.0;
}
```

```
int main() {
    sample X; // object X
    X.setvalue();
    cout << "Mean value = " << mean(X) << "\n";

    return 0;
}
```

```
#include<iostream>
#include<cstring>
using namespace std;

class Person
{
    int age;
public:
    char name[50];
    void print()
    {
        cout << name << " ";
        cout << age << endl;
    }

    void set_age(int age)
    {
        this->age = age;
    }

    int get_age()
    {
        return age;
    }
    friend void change_age(Person);
};
```

```
void change_name(Person person)
{
    person.name[0] = 'R';
}

void change_age(Person person)
{
    person.age = 31;
}

int main()
{
    Person person;
    strcpy(person.name, "Sita");
    person.set_age(30);

    change_name(person);
    change_age(person);

    person.print();

    return 0;
}
```

```
#include<iostream>
#include<cstring>
using namespace std;

class Person
{
    int age;
public:
    char name[50];
    void print()
    {
        cout << name << " ";
        cout << age << endl;
    }

    void set_age(int age)
    {
        this->age = age;
    }

    int get_age()
    {
        return age;
    }
    friend void change_age(Person);
};
```

```
void change_name(Person person)
{
    person.name[0] = 'R';
}

void change_age(Person person)
{
    person.age = 31;
}

int main()
{
    Person person;
    strcpy(person.name, "Sita");
    person.set_age(30);

    // name ramains Sita, as its pass by value
    change_name(person);
    // age ramains 30, as its pass by value
    change_age(person);

    person.print();

    return 0;
}
```

Sita 30

```
#include<iostream>
#include<cstring>
using namespace std;

class Person
{
    int age;
public:
    char name[50];
    void print()
    {
        cout << name << " ";
        cout << age << endl;
    }

    void set_age(int age)
    {
        this->age = age;
    }

    int get_age()
    {
        return age;
    }
    friend void change_age(Person &);
};
```

```
void change_name(Person &person)
{
    person.name[0] = 'R';
}

void change_age(Person &person)
{
    person.age = 31;
}

int main()
{
    Person person;
    strcpy(person.name, "Sita");
    person.set_age(30);

    change_name(person);
    change_age(person);

    person.print();

    return 0;
}
```

```
#include<iostream>
#include<cstring>
using namespace std;

class Person
{
    int age;
public:
    char name[50];
    void print()
    {
        cout << name << " ";
        cout << age << endl;
    }

    void set_age(int age)
    {
        this->age = age;
    }

    int get_age()
    {
        return age;
    }
    friend void change_age(Person &);
};
```

```
void change_name(Person &person)
{
    person.name[0] = 'R';
}

void change_age(Person &person)
{
    person.age = 31;
}

int main()
{
    Person person;
    strcpy(person.name, "Sita");
    person.set_age(30);

    // Name changes to Rita, as its pass by reference
    change_name(person);
    // age changes to 31, as its pass by reference
    change_age(person);

    person.print();

    return 0;
}
```

Rita 31

❑ Using Friend Function to Add Data Objects of two Different Classes

```
class ABC; // Forward declaration

class XYZ {
   int data;
public:
   void setvalue(int value) {
      data = value;
   }
   friend void add(XYZ, ABC); // Friend function declaration
};

class ABC {
   int data;
public:
   void setvalue(int value) {
      data = value;
   }
   friend void add(XYZ, ABC); // Friend function declaration
};
```

```
// Friend function definition
void add(XYZ obj1, ABC obj2) {
   cout << "Sum of data values of XYZ and ABC objects using friend function = "<< obj1.data + obj2.data;
}


int main() {
   XYZ X;
   ABC A;

   X.setvalue(5);
   A.setvalue(50);

   add(X, A); // Calling friend function

   return 0;
}
```

Note:-

- The function add() has arguments from both XYZ and ABC. When the function add() is declared as a friend in XYZ for the first time, the compiler will not acknowledge the presence of ABC unless its name is declared in the beginning as class ABC; This is known as ‘forward’ declaration.
- As pointed out earlier, a friend function can be called by reference. In this case, local copies of the objects are not made. Instead, a pointer to the address of the object is passed and the called function directly works on the actual object used in the all. This method can be used to alter the values of the private members of a class. Remember, altering the values of private members is against the basic principles of data hiding. It should be used only when absolutely necessary.

❑ Swapping Private Data of Classes

```
#include <iostream>
using namespace std;

class class_2; // Forward declaration

class class_1 {
   int value1;
public:
   void indata(int a) {
      value1 = a;
   }
   void display(void) {
      cout << value1 << "\n";
   }
   friend void exchange(class_1 &, class_2 &); // Friend function declaration
};

class class_2 {
   int value2;
public:
   void indata(int a) {
      value2 = a;
   }
   void display(void) {
      cout << value2 << "\n";
   }
   friend void exchange(class_1 &, class_2 &); // Friend function declaration
};
```

```
// Friend function definition
void exchange(class_1 &x, class_2 &y) {
   int temp = x.value1;
   x.value1 = y.value2;
   y.value2 = temp;
}

int main() {
   class_1 C1;
   class_2 C2;

   C1.indata(100);
   C2.indata(200);

   cout << "Values before exchange" << "\n";
   C1.display();
   C2.display();

   exchange(C1, C2); // Swapping

   cout << "Values after exchange" << "\n";
   C1.display();
   C2.display();

   return 0;
}
```

```
#include <iostream>
using namespace std;

class class_2; // Forward declaration

class class_1 {
   int value1;
public:
   void indata(int a) {
      value1 = a;
   }
   void display(void) {
      cout << value1 << "\n";
   }
   friend void exchange(class_1 &, class_2 &); // Friend
                                                function declaration
};

class class_2 {
   int value2;
public:
   void indata(int a) {
      value2 = a;
   }
   void display(void) {
      cout << value2 << "\n";
   }
   friend void exchange(class_1 &, class_2 &); // Friend
                                                function declaration
};
```

Output:-

Values before exchange
100
200
Values after exchange
200
100

```
// Friend function definition
void exchange(class_1 &x, class_2 &y) {
   int temp = x.value1;
   x.value1 = y.value2;
   y.value2 = temp;
}

int main() {
   class_1 C1;
   class_2 C2;

   C1.indata(100);
   C2.indata(200);

   cout << "Values before exchange" << "\n";
   C1.display();
   C2.display();

   exchange(C1, C2); // Swapping

   cout << "Values after exchange" << "\n";
   C1.display();
   C2.display();

   return 0;
}
```

//Like other functions main() can also be a friend of a class.

```
#include<iostream>
using namespace std;

class A
{

int pvt_member;

friend int main();

};
```

```
int main() {

    A a1;

    a1.pvt_member=10;

    cout<<a1.pvt_member;

    return 0;

}
```

//Like other functions main() can also be a friend of a class.

```
#include<iostream>
using namespace std;

class A
{

int pvt_member;

friend int main();

};
```

```
int main() {

    A a1;

    a1.pvt_member=10;

    cout<<a1.pvt_member;

    return 0;

}
```

- **Member functions of other classes can also be declared as friend functions**

```
class Y
{
        friend int X::fun();
};
```

  - Function **fun** is a **member function of class X and friend of class Y**
    - It **needs to be invoked with object of class X**
    - If **there are any objects of class Y** (within scope of **fun**), then function **fun would be able to access private members of those objects of Y**
    - **But there reverse is not true**
      - **Methods of class Y can not access private members of class X, unless those methods are declared as friend within class X**

- **Friend class**

```
class Y
{
        friend class X;
};
```

  - **All the methods of class X, would become friends of class Y**
    - So all the methods of class X can access private members of class Y
    - Methods of class Y can not access private members of class X unless
      - **Those methods are declared as a friend within class X, within the definition of X**
      - **Or if class Y has been declared as friend class of X**

  - **In other words, friend relationship is not commutative**

this pointer

The this pointer is an **implicit pointer** available inside **non-static member functions** of a class. It points to the object that invoked the function.

In simpler terms:
If object obj1 calls a member function, inside that function, this points to obj1.
So this gives access to **the current object**.

Why Needed?

- To refer to the calling object inside member functions.
- To resolve naming conflicts (especially when local variables or parameters shadow member variables).
- To return the calling object itself (used in method chaining).
- To pass the current object as a parameter to another function.

Contd.

- ***this*** keyword
  - ***this*** keyword is like **constant pointer to the object which invoked the method**
  - Hence it is **generally used with arrow operator**
    - e.g. **this->data-member = 7;**
    - e.g. **this->member-function();**
  - It can be **used only inside the methods of the class. It can not be used in independent function**
  - Can be **used to access object members which are hidden behind local variables of the same name.**
  - Methods can access other methods and data members even without use of **this** keyword (as far as members are not hidden behind local variable of the same name).

this pointer

```
class Test
{
  int mark;
  float spi;
  public:
   void SetData(){
    mark = 70;
    spi = 6.5;
   }
   void DisplayData(){
     cout << "Mark= "<<mark;
     cout << "spi= "<<spi;
   }
} ;
int main()
{
   Test o1;
   o1.SetData();
   o1.DisplayData();
}
```

this->mark = 70;
this->spi = 6.5 ;

- Within member function, the members can be accessed directly, without any object or class qualification.
- But implicitly members are being accessed using **this** pointer

- When a member function is called, it automatically passes a **pointer** to invoking object.

this pointer (Cont...)

```
class sample
{
  int a,b;
  public:
    void input(int a,int b){
      this->a = a + b;
      this->b = a - b;
    }
    void output(){
     cout<<"a = "<<a;
     cout<<"b = "<<b;
    }
};
int main()
{
    sample ob1;
    int a=5,b=8;
    ob1.input(a,b);
    ob1.output();
}
```

this pointer is used when local variable's name is same as member's name.

this pointer (Cont...)

```
class Number {
private:
    int value;
public:
    Number& setValue(int v) {
        this->value = v;
        return *this;  // returns the current object by
reference
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Number n;
    n.setValue(10).display();  // method chaining
}
```

this pointer (Cont...)

- **Important Notes:**

this is not available in:

- Static member functions (they do not belong to any object)
- Outside of class functions

this pointer (Cont...)

```
class person
{
    int age;
    public:
        person(int x){age=x;}
        void display(){cout<<"Age="<<age;}
        person olderperson(person p)
        {
            if (age > p.age)
              return *this;
            else
              return p;
        }
};
int main()
{
    person r(35),h(30);
    person o = r.olderperson(h);
    o.display();
}
```

this pointer is used to return reference to the calling object

invoking object

argument object

Return Object

```
class matrix
{
    int m[3][3];

public:
    void read(void)
    {
        cout << "Enter the elements of the 3X3 matrix:\n";
        int i, j;
        for (i = 0; i < 3; i++)
            for (j = 0; j < 3; j++)
            { cout << "m[" << i << "][" << j << "] = ";
              cin >> m[i][j];
            } }
    void display(void)
    {
        int i, j;
        for (i = 0; i < 3; i++)
        {
            cout << "\n";
            for (j = 0; j < 3; j++)
            {
                cout << m[i][j] << "\t";
            } }  }

    friend matrix trans(matrix);
};
```

```
matrix trans(matrix m1)
{
    matrix m2;  // creating an object
    int i, j;
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            m2.m[i][j] = m1.m[j][i];

    return (m2);  // returning an object
}

int main()
{
    matrix mat1, mat2;
    mat1.read();
    cout << "\nYou entered the following matrix:";
    mat1.display();

    mat2 = trans(mat1);
    cout << "\nTransposed matrix:";
    mat2.display();

    return 0;
}
```

When a function **returns an object**, it gives you an **instance of a class**, either:

- **By value** – returns a **copy** of the object.
- **By reference** – returns a **reference to the same object** (no copy).
- **By pointer** – returns the **address of the object**.

Return Type	Meaning	Copy Made?	Example Use Case
Student	Returns a **copy** of object	✅ Yes	Temporary result needed
Student&	Returns a **reference** to same object	❌ No	Method chaining, in-place update
Student*	Returns a **pointer** to the object	❌ No	Dynamically created object

1. Returning an Object by Value (Copy)

```
class Student {
    int id;
public:
    Student(int x) { id = x; }

    Student getStudent() {
        return *this; // returns a COPY of the
current object
    }

    void show() {
        cout << "ID: " << id << endl;
    }
};
```

return *this; returns a copy of the current object

So when you do:

```
Student s1(1);
Student s2 = s1.getStudent();
// s2 is a separate copy
```

Changes in s2 won't affect s1.

2. Returning an Object by Reference (No Copy)

```
class Student {
    int id;
public:
    Student(int x) { id = x; }

    Student& updateId(int newId) {
        id = newId;
        return *this; // returns current
object (by reference)
    }

    void show() {
        cout << "ID: " << id << endl;
    }
};
```

```
Student s1(10);
s1.updateId(99).show(); // Chaining
possible
```

✅ updateId() returns the same object — no copy.
✅ Useful for method chaining and in-place modifications

3. Returning an Object by Pointer

```
class Student {
    int id;
public:
    Student(int x) { id = x; }

    Student* getPointer() {
        return this; // returns address of
current object
    }

    void show() {
        cout << "ID: " << id << endl;
    }
};
```

```
Student s1(5);
Student* ptr = s1.getPointer();
ptr->show(); // Access using pointer
```

✅ Useful when object needs to be passed to a function expecting a pointer.

Feature	Student	Student&	Student*
Copy created?	✅ Yes	❌ No	❌ No
Supports chaining	❌ No	✅ Yes	❌ Not directly
Can be nullptr?	❌ No	❌ No	✅ Yes
Used for dynamic object?	❌ Rare	❌ Rare	✅ Common

const member functions

- If member function does not alter any data member of the object then it may be declared as const method
- const member functions does not change the state of the object
- Keyword const is appended at the end of the function prototype (**both in declaration and definition**)
- Compiler will generate error if const member function tries to alter state of the object
- Though such function can have local variables and it can also **alter local variable**.
- Benefit of const member functions
  - Developer will not modify state of the object from const member functions by mistake
  - We will see later that const object can call const member functions, but it can not call other non-const member functions

Example:

```cpp
#include <iostream>
using namespace std;

class MyClass {
    int value;

public:
    MyClass(int v) : value(v) {}

    // Const member function
    int getValue() const {
        // value = 100; // ❌ ERROR: can't modify value
        return value;
    }

    // Non-const member function
    void setValue(int v) {
        value = v; // ✅ Allowed
    }
};
```

```cpp
int main() {
    MyClass obj(10);

    cout << obj.getValue() << endl; // Works fine
    obj.setValue(20);
    cout << obj.getValue() << endl;
}
```

Note :-

```
#include<iostream>
using namespace std;

void fun(int &i)
{
    i = 7;
    std::cout << i << std::endl;
}

class Test
{
    int x = 5;
public:
    void abc()
    {
        // passing private member to non-member function as reference

        fun(x);
        std::cout << x << std::endl;
    }
};

int main()
{
    Test t;
    t.abc();
    return 0;
}
```

```
7
7
```

- **This is valid and does not violate the rules of encapsulation (data hiding)**, because **member function by itself (knowingly) is sending the private member to non-member function**
- Member function is knowingly compromising its privacy, and it is allowed
- We have been doing something similar all along, when we use cin to scan the values of private members

Pointers to Members

A **pointer to a member** does **not** store a memory address of a variable or function directly.
Instead, it stores the **offset/position** of a member **within a class**, and you must use an object (or pointer to an object) to access the actual value.

Think of it as:
"A bookmark that tells you where a member lives inside any object of this class."

<u>Syntax:</u>

For **Data Members**:

```
data_type ClassName::* pointerName;
```

For **Member Functions**:

```
return_type (ClassName::* pointerName)(parameter_list);
```

Blueprint Analogy for Pointers to Members

- **Class** → The **blueprint** of a house. It describes where each room (member) is, but there's no actual building yet.
- **Member variable** → A **specific room** on the blueprint (like the kitchen).
- **Pointer to member** → A **mark on the blueprint** saying, "This is where the kitchen is, measured from the front door." It's not the kitchen itself — just the location.
- **Object (instance)** → The **real, built house** based on the blueprint.
- **Accessing through pointer to member** → Walking into a specific house and using your marked spot on the blueprint to find that exact room.

- Without a real building (object), your mark (pointer to member) is useless.
- The mark works on **any house built from that blueprint** (any object of that class).
- You still need the correct **path inside the house** (.* or ->*) to get to that room.

Example — Pointer to Data Member

```cpp
#include <iostream>
using namespace std;

class MyClass {
public:
    int x;
    double y;
};
```

```cpp
int main() {
    MyClass obj;
    obj.x = 10;
    obj.y = 3.14;

    // Pointer to data member 'x'
    int MyClass::* ptr = &MyClass::x;

    // Access using object
    cout << "x = " << obj.*ptr << endl;

    // Access using pointer to object
    MyClass* pObj = &obj;
    cout << "x (via pointer) = " << pObj->*ptr << endl;

    // Modify value
    obj.*ptr = 50;
    cout << "Modified x = " << obj.x << endl;
}
```

Output:-

```
x = 10
x (via pointer) = 10
Modified x = 50
```

Example — Pointer to Member Function

```cpp
#include <iostream>
using namespace std;

class MyClass {
public:
    void show() {
        cout << "Hello
from show()" << endl;
    }
};
```

```cpp
int main() {
    MyClass obj;

    // Pointer to member function 'show'
    void (MyClass::* fptr)() = &MyClass::show;

    // Call using object
    (obj.*fptr)();

    // Call using pointer to object
    MyClass* pObj = &obj;
    (pObj->*fptr)();
}
```

Output:-

```
Hello from show()
Hello from show()
```

Key Points to Remember :-

1. Special operators:
   - **.*** → Use with object.
   - **->*** → Use with pointer to object.
2. Pointer to member **needs a class instance** to be used — it's not like a regular pointer holding a direct memory address.
3. Syntax is trickier than normal pointers, so parentheses matter when calling member functions.
4. Can be used with arrays of pointers to members, function callbacks, or table-driven code.

Local classes

A **local class** is a class **defined inside a function**, rather than at the global or namespace level. It only exists **within the scope of that function**.

Key Points about Local Classes :-

1. **Scope** – Visible only inside the function/block where they are defined.
    - Outside that function, the class does not exist.
2. **Member Functions** – Must be defined inside the class (no separate definition outside, unless you fully qualify inside the same function).
3. **Access to Variables** –
    - Can access static variables of the enclosing function.
    - Cannot directly access non-static local variables of the enclosing function unless they are static or captured in C++11 lambdas.
4. **Static Members** – Local classes cannot have static data members (except static const integral types).
5. **Friendship & Nesting** – They can have friends and can be used with nested classes, but still bound by scope rules.

Example:

```
#include <iostream>
using namespace std;

void demo() {
    static int count = 0; // static local variable

    class LocalClass {
    public:
        void showCount() {
            cout << "Count = " << count << endl; // ✅ allowed
        }
    };

    LocalClass obj;
    obj.showCount();
    count++;
}

int main() {
    demo();
    demo();
}
```